

GestureFlow

User guide and implementation notes

GestureFlow turns one visible hand into Windows mouse controls and a hold-to-talk trigger for FlowSpeak. The camera tracks your hand locally. Your existing FlowSpeak installation continues to own microphone capture, transcription, settings, and text insertion.

Start here

1. After setup, launch the GestureFlow desktop shortcut or `Start-GestureFlow.cmd`. The original PC may also use its existing `GestureFlow.exe`.
2. Start FlowSpeak normally or use Launch in the Connected to FlowSpeak card. Confirm its existing Right Control hold-to-talk shortcut works in a text editor.
3. Select your controlling hand and Pointer display. The camera opens automatically. To choose a different Camera index, stop the camera, change the index, then start it again.
4. Enable Preview only - do not control PC to practice without moving the mouse or pressing keys.
5. Calibrate with your elbow resting comfortably. Disable Preview only when ready, then hold an open palm for about 0.35 seconds to turn gesture control ON.
6. Extend only the index to point at a text field. Keep it straight and touch the thumb to its side to click. Touch middle fingertip to thumb, hold while speaking, then separate to finish.

Keep your keyboard and physical mouse available while learning. `Ctrl+Alt+G` locks gesture actions globally. Use the same shortcut or the app's unlock control to leave that lock, then show an open palm to turn control ON.

Open palm turns control ON; a closed fist turns it OFF. Camera tracking continues in both states. Index pointing moves the pointer only when control is ON; it never turns control ON or OFF. Gestures cannot bypass a keyboard lock. Stop the camera or close GestureFlow to end capture.

Right-hand control is selected by default. Hand labels are corrected for the mirrored camera input so Right means your anatomical right hand. Tracking can detect both hands, but only the selected hand controls input; your left hand can rest on your face. A right-hand label must have at least 80% confidence. If missing or uncertain, GestureFlow never substitutes the left: held input releases and movement freezes, while the ON/OFF state is retained. The selector remains available to change your preference.

Install on a laptop

GestureFlow v0.1.1 targets Windows on Intel/AMD 64-bit processors. This release is not for macOS, Linux, or native Windows ARM. Install Python 3.13 64-bit first; choose that version on python.org's Windows downloads page. Internet access is required during setup.

1. Extract the complete source ZIP into a writable folder such as Documents\GestureFlow. Do not run from inside the ZIP or place it in Program Files.
2. Double-click Install.cmd. It creates a local .venv, installs pinned binary wheels, runs pip check, validates model loading and the input helper, and creates a desktop shortcut.
3. Launch the shortcut or Start-GestureFlow.cmd. This route needs neither the optional GestureFlow.exe nor .NET.
4. Permit camera access for desktop apps in Windows privacy settings. Select the correct camera index if needed.
5. Install FlowSpeak separately and confirm physical Right Ctrl works. GestureFlow looks in the current user's Documents folders; use Locate to select the laptop's FlowSpeak.exe when necessary.
6. Calibrate your comfortable range on the laptop, practice in Preview only, then turn control ON with an open palm.

The ZIP includes source, model, docs, and setup scripts. Python, installed dependencies, and FlowSpeak are not bundled. Do not copy another PC's .venv, settings.json, or calibration. Fresh settings use Right as the controlling hand with the floating status overlay off. Keep the extracted folder intact after installation.

The dashboard fits the available screen at startup and provides horizontal and vertical scrolling on small or scaled displays; both scrollbars were verified in an offscreen 640 x 480 check.

If setup fails, read the setup window and rerun Install.cmd after correcting the issue. For an incompatible existing environment, extract into a fresh folder. To start from a terminal instead, run `.\.venv\Scripts\python.exe main.py` from a terminal in the project folder. See docs\Laptop-Install.md for the dedicated quickstart and troubleshooting.

Setup checks on the development PC do not establish compatibility with your physical laptop's camera, microphone, permissions, or FlowSpeak configuration. Perform the practical gesture and dictation checks on that laptop.

Gesture controls

Action	Hand position	Behavior
Move pointer	Extend only index finger; fold other three	Move your index fingertip within your calibrated area.
Turn control ON	Open palm	Hold about 0.35 seconds. Camera remains on; pointer stays still until you point.
Click	Keep index straight; touch thumb to its side	Presses left mouse button at the current pointer; release contact to complete the click.
Drag	Hold thumb against side of straight index	After 0.24 seconds, move the index fingertip; release contact to drop.
FlowSpeak	Bend middle to thumb; extend index, ring, pinky	Hold this exact shape for Right Control. Release or change shape to finish recording.
Enter / send	Bend ring to thumb; extend index, middle, pinky	Hold 0.18 seconds for one Enter. Release or change shape before repeating.
Scroll	Thumbs-up: curl all four fingers, extend thumb	Hold about 0.16 seconds, then move up/down. Return to index pointing to move the cursor.
Right-click	Extend thumb and pinky; fold other three fingers	Hold the shaka gesture for about 0.30 seconds. One click per gesture.
Switch window	Extend index and pinky; close middle, ring, thumb	Hold to open switcher and cycle windows. Release gesture to select the highlight.
Turn control OFF	Closed fist with thumb tucked	Hold about 0.30 seconds. Gesture actions stop; app and camera stay on.
Lock controls	Ctrl+Alt+G	Locks input globally; gestures cannot unlock this state.
Lock in app	Escape with GestureFlow focused	Locks input.

To click, keep the index extended and the other fingers curled, like a finger gun. Touch the thumb to the side of the index, not its fingertip. The old index-tip pinch no longer clicks. For FlowSpeak, bend middle to thumb with index, ring, and pinky extended. For Enter, bend ring to thumb with index, middle, and pinky extended. Maintain those exact shapes; losing contact or the finger pattern releases after about 0.10 seconds. A tucked-thumb fist ends held input and turns control OFF.

How the interaction feels

While ON, index pointing maps your fingertip to the monitor. Separate horizontal and vertical sensitivities control travel; smoothing reduces jitter. The pointer freezes as the thumb approaches the index side. Hold contact to drag using the fingertip.

Control stays ON through all actions. The pointer moves only while pointing or dragging. Hand loss releases input and freezes movement without switching control OFF. Returning with a valid action needs no activation gesture.

Adjusting the controls

Setting	What it changes	Practical adjustment
Camera	Numeric camera device index	Start at 0; try another index if the preview is wrong.
Controlling hand	Which detected hand drives input	Defaults to Right. The left hand is ignored even when both hands are visible.
Target monitor	Display used for pointer mapping	Choose the display on which you want to work.
Smoothing	How strongly movement is filtered	Increase for steadier motion; decrease for faster response.
Horizontal sensitivity	Sideways gain; 50-300%	Higher means less fingertip travel; 100% uses your full calibrated width.
Vertical sensitivity	Up/down gain; 50-400%	Higher means less vertical travel; 100% uses your full calibrated height.
Pinch sensitivity	Distance required to recognize a pinch	Adjust if a natural touch is missed or near-touches trigger.
Scroll speed	Wheel response to vertical movement	Start low and increase gradually.
Preview only	Allows tracking but suppresses input	Use while calibrating or practicing gestures.

Sit comfortably with your whole hand visible and a simple background. Face your palm toward the camera; steep side angles and overlapping fingertips make recognition less reliable. Light your hand from the front and avoid a bright window behind it.

The pointer area in the preview shows the part of the camera image mapped to the full display. Keep your fingertip in that area; you do not need to cover the whole camera view. Adjust horizontal and vertical sensitivity independently after calibration. Increase only vertical sensitivity if reaching the top or bottom still needs too much movement.

You can also click or drag in the camera preview to reposition the pointer area directly. The default area sits toward the bottom-right of the preview. Its center is limited automatically so the entire area remains visible. Adjust in Preview only or with gesture controls locked so hand movement does not interfere with your physical mouse.

Calibrate your comfortable range

Rest your elbow where you intend to use the app. Click Calibrate wrist range and extend only your index finger. Follow the five prompts: center, comfortable left, comfortable right, comfortable up, and comfortable down. Hold each position for three seconds; avoid stretching to reach the camera's edges. If the hand is lost or the pointing pose changes, that step's hold restarts.

Calibration maps your comfortable fingertip limits to the screen, saves them, and sets both sensitivities to 100%. Input is suppressed throughout; the prior ON/OFF state is restored afterward. Cancel preserves the prior range. If previously OFF, show an open palm afterward. A range that is too small or reversed, or a center outside the extremes, is rejected. Retry from the same resting position.

FlowSpeak integration: existing behavior preserved

GestureFlow does not replace or modify FlowSpeak. It synthesizes the same Right Control key-down and key-up sequence used by FlowSpeak's existing hold-to-talk shortcut. Middle-thumb contact starts the hold; separation releases it. FlowSpeak must already be running and configured for that shortcut.

Select the destination text field before recording. GestureFlow does not choose where the transcript goes. FlowSpeak's existing permissions, microphone selection, account/API configuration, network requirements, transcription behavior, and insertion behavior still apply.

To send a dictated chat, release the middle-thumb pinch and wait until the text appears. Then briefly hold ring-thumb contact to tap Enter. GestureFlow does not wait for transcription or queue a send action. Enter follows the focused app's normal behavior: it may send a message, insert a newline, or activate a control. Release physical modifier keys before using this gesture; if one is held, GestureFlow blocks Enter and asks you to release and retry.

Hold the window-switch pose for 0.30 seconds to open Alt+Tab. Alt stays held; Tab advances every 0.75 seconds. Release the pose for 0.15 seconds to release Alt and select the highlight; cycling stops during that release delay. Control must be ON and physical modifiers released. Hand loss, fist, lock, or exit also releases Alt.

Hand loss for about 0.30 seconds releases held input and freezes movement, retaining the ON/OFF state. Releasing a FlowSpeak hold may finish and transcribe the recording; it does not cancel it.

Avoid using physical Right Control at the same time as a gesture hold. GestureFlow checks whether that key is already down before starting its own hold. Right Control is an ordinary modifier key, so typing or clicking while dictating can have the same shortcut effects as holding it physically. If FlowSpeak is not detected as running, GestureFlow asks you to launch it instead of sending a new modifier hold. Running status confirms the process exists, not microphone or transcription readiness.

Use Locate in the FlowSpeak card if the executable has moved. Show floating status controls the overlay, which does not take keyboard focus. Guides opens the documentation folder. Settings save automatically. Reset tuning keeps your calibration and restores smoothing 45, horizontal sensitivity 100%, pinch sensitivity 30, and scroll speed 100%. Vertical sensitivity resets to 100% with a saved range, or 150% before calibration.

Architecture and privacy

The app is implemented in Python with a PySide6/Qt desktop interface, keeping the camera, state machine, and UI in one runtime. The thin .NET launcher only starts that application. This replaces the initial WPF proposal without changing FlowSpeak.

Camera frames enter an OpenCV capture loop. MediaPipe Hand Landmarker estimates 21 hand landmarks using the official float16 v1 task model. GestureFlow calculates palm position, finger extension, and fingertip distances, then passes them through a deterministic gesture state machine. This build uses the Hand Landmarker and its own gesture rules, rather than MediaPipe's pretrained Gesture Recognizer. A pointer mapper applies screen bounds and smoothing. A separate input broker sends Windows mouse and keyboard events.

The broker owns injected holds and monitors a heartbeat. If tracking or the UI stops communicating for roughly 0.65 seconds, it releases held input. Normal pause, camera stop, hand loss, and exit also release holds. This limits stuck-input risk; the keyboard lock remains the user's immediate override.

Video processing is local. GestureFlow does not record camera video or upload camera frames. It does not capture microphone audio. FlowSpeak retains its existing cloud transcription path and privacy behavior. Initial dependency/model downloads require internet access; hand tracking uses the locally installed model afterward.

Troubleshooting

Symptom	What to check
Camera will not open	Check Windows camera privacy access for desktop apps; close other camera apps; try the correct camera index.
Preview works but cursor does not move	Check Preview only and keyboard lock. Show an open palm to turn control ON, then point with only the index.
Left hand is visible but does nothing	This is intentional with Right selected. Keep the right hand visible and face its palm toward the camera.
Pointer shakes	Improve lighting, keep the entire hand visible, and increase smoothing.
Cursor requires too much arm travel	Calibrate with your elbow resting, then increase horizontal or vertical sensitivity as needed.
You must raise your arm too high	Recalibrate using a comfortable up/down range and increase vertical sensitivity. Drag the preview area lower if needed.
Click is missed	Keep index straight and touch its side with the thumb; a fingertip pinch no longer clicks.
FlowSpeak/Enter shape is missed	Bend only the target finger to the thumb and extend the other three. Compact pinches are accepted. Maintain the shape; changing it releases after about 0.10 seconds.
Scroll does not start	Curl all four fingers and extend your thumb upward; hold briefly, then move vertically. Tucking the thumb changes to fist pause.
Right-click repeats poorly	Leave the shaka gesture before making it again; hold it steadily for about 0.30 seconds.
Dictation does not start	Confirm FlowSpeak is running and physical Right Control works first. Check its own microphone/transcription settings.
Dictation ends unexpectedly	Keep the hand in frame and the pinch visible; hand loss releases the hold.
Input fails in an administrator app	Windows may block injected input into an elevated target. Test in a normal text editor first.
Ctrl+Alt+G is unavailable	Another app may own this system shortcut. Use Lock controls or Escape while GestureFlow is focused; the app displays a warning if registration fails.

Scope and limits

This release supports one controlling hand, a selected monitor, pointer movement, left click/drag, scrolling, right-click, Enter, window switching, ON/OFF control, and FlowSpeak hold-to-talk. Precision mode, a tool launcher, app-specific profiles, and automatic mouse takeover are not part of this build. Gestures use landmark geometry rather than a custom-trained classifier.

Camera angle, lighting, motion blur, hand occlusion, and individual hand posture affect accuracy. Stability thresholds add small intentional delays. Software checks do not establish comfort or reliability for your actual hand and camera; use the practical checklist below for that acceptance test.

Your first practical test

1. In Preview only, try every pose and confirm the status matches your intention.
2. Calibrate your comfortable range. Show an open palm to turn control ON, then point with only the index and reach each screen corner. Adjust sensitivity.
3. In a disposable document, click with thumb-to-index-side contact, select text by holding and dragging, then release. Confirm no button remains held.
4. Open a long page, scroll both ways, then open and dismiss a context menu using shaka.
5. Turn control OFF with a fist; pointing alone should do nothing. Turn it ON with open palm. Lock with Ctrl+Alt+G and verify no gesture unlocks it.
6. In a text field with FlowSpeak running, hold middle-thumb contact, speak a short sentence, and release. Confirm normal transcription/insertion.
7. Move your hand out of frame during a short hold. Confirm release and frozen movement, then return and point; control should still be ON.

Developer reference

On each computer, the application lives in the folder where you extracted it, typically your Documents\GestureFlow folder. The original development PC used C:\Users\MUN86606\Documents\GestureFlow; that historical path is not required on a laptop. Both launch methods use the project-local Python environment; keep the folder and .venv together.

Installed runtime: Python 3.13.13, PySide6 6.11.2, MediaPipe 1.0.1, opencv-contrib-python 5.0.0.93, psutil 7.2.2, and NumPy 2.5.3. Windows input uses ctypes and Win32 APIs. The MediaPipe Hand Landmarker task asset is stored in models\hand_landmarker.task. The C# launcher targets .NET 10 for Windows and is published self-contained; the Python application still uses .venv.

File or directory	Responsibility
Start-GestureFlow.cmd	Included launcher for the local Python app; no .NET or VBScript required.
Install.cmd / Install.ps1	Windows/Python checks, local environment setup, pinned dependencies, shortcut creation.
scripts\verify_install.py	Import, model, and input-helper setup validation.
GestureFlow.exe	Optional original-PC native launcher; not needed for laptop setup.
main.py	Python entry point.
gestureflow\app.py	PySide6 window, settings, preview, floating status, FlowSpeak process detection, input dispatch.
gestureflow\tracker.py	Camera acquisition, mirrored preview, up to two detected hands, confidence-gated controlling-hand selection.
gestureflow\selection.py	Selects the requested hand from detections; rejects uncertain handedness.
gestureflow\engine.py	Gesture classification, hold/release state, arbitration, timing, pointer mapping.
gestureflow\calibration.py	Five-step fingertip sampling and comfortable-range validation.
gestureflow\native.py	Typed Win32 mouse, key, cursor, DPI, and monitor calls.
gestureflow\broker.py	Separate input owner, heartbeat handling, stale-command rejection, release cleanup.
models\	Locally stored hand-landmark model.
tests\	Deterministic engine and broker checks without live input.
docs\	This guide, the separate cheat sheet, and editable Markdown sources.
.venv\	Installed Python runtime packages for the project.
requirements.txt	Pinned direct runtime dependencies.
requirements-lock.txt	Snapshot of all installed package versions.
settings.json	Saved tuning, camera/hand/display choices, overlay choice, and FlowSpeak path.
launcher\	C# launcher source and project.
work\	Diagnostic logs, status snapshot, process lock, and build intermediates.

Developer commands

From a PowerShell terminal in the project folder, run automated checks with:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Run the application directly:

```
.\.venv\Scripts\python.exe main.py
```

For tracking practice without injection, add `--preview`. To open the UI without starting camera capture, add `--no-camera`. Both flags may be combined. Close the existing instance before starting another; the app prevents duplicate instances.

To recreate a missing environment, install Python 3.13 for Windows and run this from the project folder:

```
py -3.13 -m venv .venv
```

Restore the recorded dependency versions:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
```

For only the direct pinned dependencies instead:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

To rebuild the native launcher with a .NET 10 SDK, publish from the project folder:

```
dotnet publish launcher\GestureFlow.csproj -c Release -r win-x64 --self-contained true  
-p:PublishSingleFile=true -o work\launcher-publish
```

Copy the published `GestureFlow.exe` to the project root after closing the app. The executable locates `main.py` and `.venv` relative to its own directory.

For troubleshooting, inspect `work\gestureflow.log` and `work\input-helper.log`. `work\status.json` is a periodically refreshed health snapshot with camera activity, frames, mode, pause/lock state, FlowSpeak detection, and helper status. It does not contain camera frames or transcripts.

Verification

The automated suite contains 83 passing tests. It covers calibration, fingertip pointing, pinch exclusivity, click/drag/Enter, hold release, hand loss, processing stalls, ON/OFF control, lock, right-click repeat protection, scrolling, monitor bounds, stale commands, EOF, broker watchdog cleanup, and confident right-hand selection regardless of result order. These tests use constructed hand states and a fake input backend; they do not prove physical-camera gesture accuracy or successful FlowSpeak transcription.

On the original PC, the native launcher built and the UI was inspected. Camera 0 and inference ran near 30 FPS; FlowSpeak was detected. `Ctrl+Alt+G` locked and unlocked controls while capture continued. The shortcut uses Windows `RegisterHotKey` and Qt native events. FlowSpeak files were not modified.

Physical gesture accuracy, comfortable pointer tuning, and the complete spoken-dictation-to-text flow remain for your hands-on test. The software checks above establish startup, camera processing, interface availability, and deterministic input behavior; they do not replace that practical validation.

Fresh setup on the development PC passed dependency, model/input-helper, and offscreen dashboard checks. This was a separate clean environment, not the physical laptop; laptop hardware and end-to-end dictation remain to be tested there.